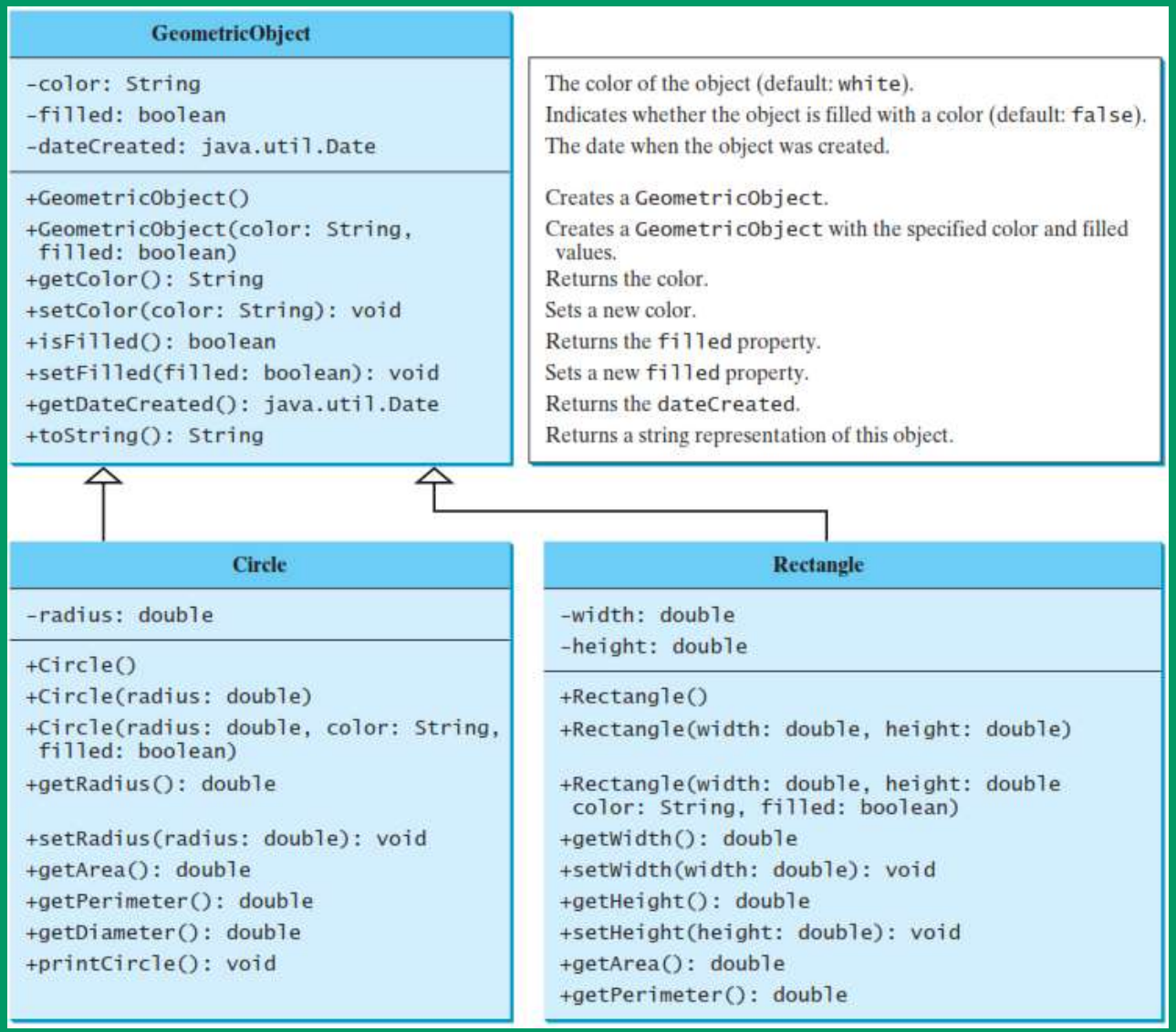
Bölüm 11 Miras ve Kapsülleme

Bir Alt Sınıf Tanımlama

Bir alt sınıf üst sınıftan özellikler ve yöntemler alır.
Ayrıca şunları yapabilirsiniz:

- Yeni özellikler ekleyin
- Yeni yöntemler ekleyin
- Üst sınıfın yöntemlerini geçersiz kılın

Üst Sınıflar ve Alt Sınıflar



GeometrikNesne1

Daire4

Dikdörtgen1

TestDaireDikdörtgen

Çalıştır

Üst sınıfın Yapıcısı

Devralındı mı?

Hayır. Miras alınmazlar.

Onlar açıkça veya örtük olarak çağrılır.

Açıkça süper anahtar kelimesi kullanılarak.

Bir yapıcı, bir sınıfın bir örneğini oluşturmak için kullanılır.
Özellikler ve yöntemlerin aksine, bir üst sınıfın yapıcıları alt sınıfta miras alınmaz. Sadece alt sınıfların yapıcılarından çağrılabilir,

anahtar kelime süper kullanılarak. *Eğer anahtar kelime süper açıkça kullanılmazsa, üst sınıfın argüman almayan yapıcısı automatik olarak çağrılır.*

Anahtar Kelime Kullanarak süper

Anahtar kelime süper, üst sınıfı ifade eder. sınıfında süper göründüğü. Bu anahtar kelime iki şekilde kullanılabilir:

- Bir üst sınıf yapıcısını çağırmak için
- Bir üst sınıf yöntemini çağırmak için

```
public Circle4(double radius, String color, boolean filled) {
  super(color, filled);
  this.radius = radius;
}
```

DİKKAT

- super anahtar kelimesini çağırmak için kullanmalısınız üst sınıf yapıcısı. Bir superclass constructor'ın adı bir alt sınıf bir sözdizimi hatasına neden olur.
- Java, kullanılan ifadeyi gerektirir anahtar kelime super, yapıcıda ilk olarak görünür.

Geçersiz Kılma vs. Aşırı Yükleme

```java
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // Bu yöntem B'deki yöntemi geçersiz kılar
  public void p(double i) {
    System.out.println(i);
  }
}
```

```java
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // Bu yöntem B'deki yöntemi aşırı yükler
  public void p(int i) {
    System.out.println(i);
  }
}
```

Object içindeki toString() yöntemi

toString() yöntemi,
nesnenin bir dize temsilini döndürür. Varsayılan uygulama, bir nesnenin örneği olduğu bir sınıf adının, @ işareti (@), ve bu nesneyi temsil eden bir sayı.

```
Loan loan = new Loan();
System.out.println(loan.toString());
```

Kod, Loan@15037e5 gibi bir şey gösterir. Bu mesaj çok yardımcı veya bilgilendirici değildir. Genellikle, toString yöntemini geçersiz kılmalısınız, böylece nesnenin sindirilebilir bir dize temsilini döndürür.

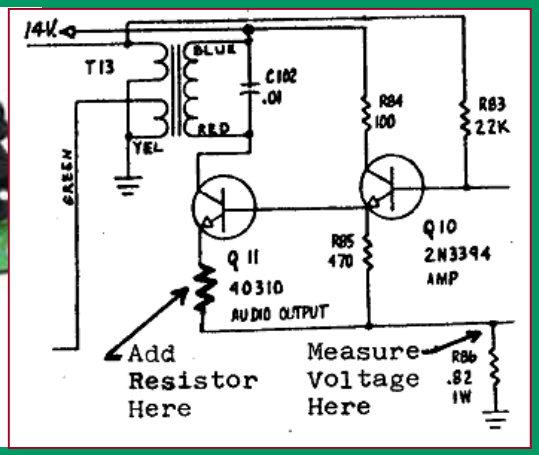
instanceof Operatörü

Bir nesnenin bir sınıfın örneği olup olmadığını test etmek için `instanceof` operatörünü kullanın:

```
Object myObject = new Circle();
// Bazı kod satırları
/**
myObject bir Circle örneği ise döküm yapın
*/
if (myObject instanceof Circle) {
   System.out.println("myObject bir Circle örneğidir");
   ...
}
```

Kapsülleme

- kapsülleme: Uygulama detaylarını müşterilerden gizleme.
  - Kapsülleme *soyutlamayı* zorlar.
    - dış görünümü (davranış) içsel görünüm (durum)
    - bir nesnenin verilerinin bütünlüğünü korur



Özel alanlar

Dışarıdan erişilemeyen bir alan

sınıf

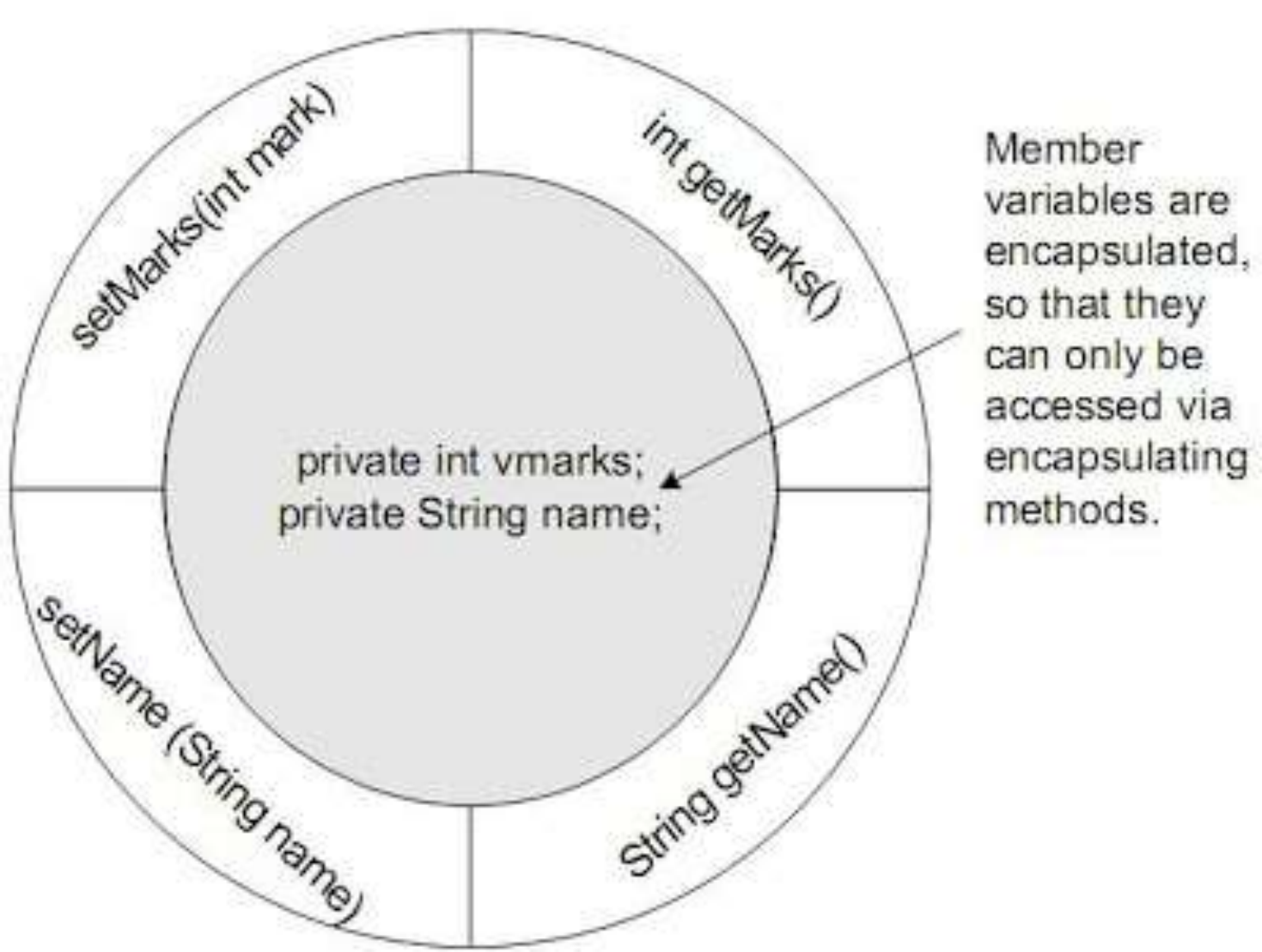
özel tür adı;

– Örnekler:

```
private int id;
özel String isim;
```

☞ Müşteri kodu, özel alanlara erişirse derlenmeyecek alanlar:

Özel duruma erişim



Özel duruma erişim

```
// x alanına "salt okunur" erişim
public int getX() {
    return x;
}
// Müşterilerin x alanını değiştirmesine izin verir
public void setX(int newX) {
    x = newX;
}
```

– Müşteri kodu daha çok şöyle görünecek:

```
System.out.println(p1.getX());
p1.setX(14);
```

Örnek: Nokta sınıfı

```
// Bir Nokta nesnesi (x, y) konumunu temsil eder.
public class Point {
    private int x;
    private int y;

    public Point(int initialX, int initialY) {
        x = initialX;
        y = initialY;
    }

    public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }

    public double distanceFromOrigin() {
        return Math.sqrt(x * x + y * y);
    }

    public void setLocation(int newX, int newY) {
        x = newX;
        y = newY;
    }

    public void translate(int dx, int dy) {
        setLocation(x + dx, y + dy);
    }
}
```

Encapsulation'ın Faydaları

- Nesne ile istemciler arasındaki soyutlama
- İstenmeyen erişimden nesneyi korur
  - Örnek: Hesap'ının bakiyesini sahte bir şekilde ar tırmak mümkün değildir.
- Nesnelerin durumunu kısıtlayabilir (**invariants**)
  - Örnek: Sadece Hesaps ile izin verin- negatif bakiye.
  - Örnek: Sadece 1-12 arasındaki bir ay ile Tarihs'y e izin verin.

ArrayList Sınıfı

Bir nesne depolamak için bir dizi oluşturabilirsiniz. Ancak dizi oluşturulduğunda boyutu sabittir. Java, ArrayList sınıfını sağlar ki bu da sonsuz sayıda nesneyi depolamak için kullanılabilir.

java.util.ArrayList	
+ArrayList()	Boş bir liste oluşturur.
+add(o: Object) : void	Bu listeye sonuna yeni bir eleman o ekler.
+add(index: int, o: Object) : void	Bu listeye belirtilen indeksde yeni bir eleman o ekler.
+clear(): void	Bu listedeki tüm öğeleri kaldırır.
+contains(o: Object): boolean	Bu liste, o öğesini içeriyorsa true döner.
+get(index: int) : Object	Belirtilen indeksteki bu listeden öğeyi döner.
+indexOf(o: Object) : int	Bu listedeki ilk eşleşen öğenin indeksini döner.
+isEmpty(): boolean	Bu liste hiçbir öğe içermiyorsa true döner.
+lastIndexOf(o: Object) : int	Bu listedeki son eşleşen öğenin indeksini döner.
+remove(o: Object): boolean	Bu listeden o öğesini kaldırır.
+size(): int	Bu listedeki öğe sayısını döner.
+remove(index: int) : Object	Belirtilen indeksteki öğeyi kaldırır.
+set(index: int, o: Object) : Object	Belirtilen indeksteki öğeyi ayarlar.

ArrayList Örneği

TestArrayList

Çalıştır

Korunan Değiştirici

Korunan değiştirici veriye uygulanabilir
ve bir sınıfta yöntemler. Korunan bir veri veya bir korumalı yöntem, aynı paket içindeki herhangi bir sınıf veya alt sınıfları tarafından erişilebilir.

Alt sınıflar bir farklı paket.

özel, varsayılan, korumalı, genel

Erişim Özeti

Üye değiştirici bir sınıfta	Aynı sınıftan erişildi	Erişim sağlandı şuradan aynı paket	Erişim sağlandı bir \ alt sınıftan	Farklı paketten erişildi
kamusal	✓	✓	✓	✓
korumalı	✓	✓	✓	–
varsayılan	✓	✓	–	–
özel	✓	–	–	–

Görünürlük Modifikatörleri

package p1;

```
public class C1 {
  public int x;
  protected int y;
  int z;
  private int u;

  protected void m() {
  }
}
```

```
public class C2 {
  C1 o = new C1();
    o.x'e erişebilir;
    o.y'ye erişebilir;
    o.z'ye erişebilir;
    o.u'ya erişemez;

    o.m()'yi çağırabilir;
}
```

package p2;

```
public class C3
          extends C1 {
    x'e erişebilir;
    y'ye erişebilir;
    z'ye erişebilir;
    u'ya erişemez;

    m()'yi çağırabilir;
}
```

```
public class C4
          extends C1 {
    x'e erişebilir;
    y'ye erişebilir;
    z'ye erişemez;
    u'ya erişemez;

    m()'yi çağırabilir;
}
```

```
public class C5 {
  C1 o = new C1();
    o.x'e erişebilir;
    o.y'ye erişemez;
    o.z'ye erişemez;
    o.u'ya erişemez;

    o.m()'yi çağırmaz;
}
```

NOT

final modifikatörü, bir metodun içindeki yerel

değişkenler üzerinde de kullanılabilir. Bir final yerel

değişken, bir metodun içinde bir sabittir.

final Modifikatörü

- Son sınıf uzatılamaz:

```
final class Math {
  ...
 }
```

- `final` değişken bir sabittir:

```
final static double PI = 3.14159;
```

- `final` yöntem alt sınıfları tarafından geçersiz kılınamaz.